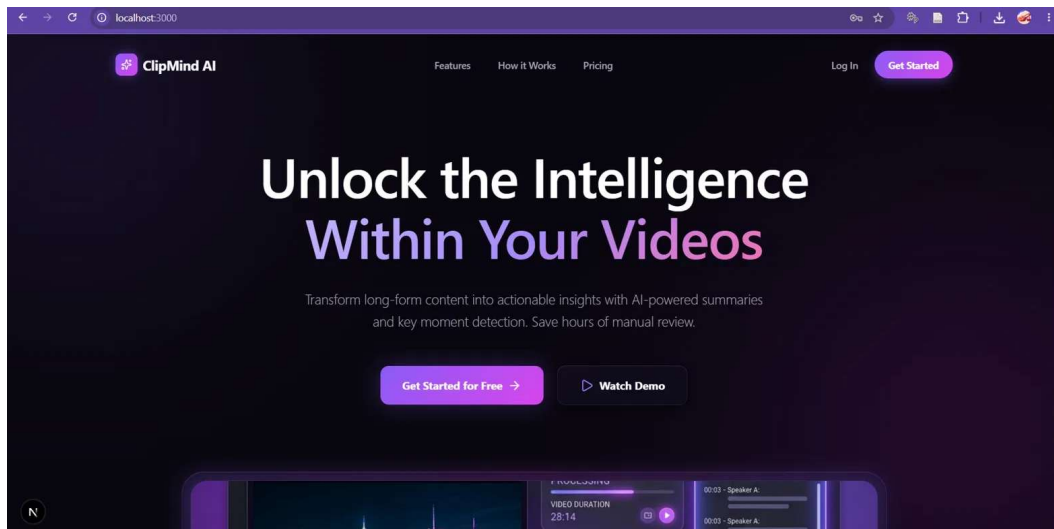


PROJECT REPORT

ClipMind AI

An AI-Powered Video Summarization & Key Moments Detection Platform



Devanshi Malhotra
ARTIFICIAL INTELLIGENCE INTERN

August 2026
Infosys Springboard Internship

Abstract

Long-form video has become the default medium for lectures, product walkthroughs, meetings, and research talks, but it remains one of the hardest formats to search or revisit. Finding a single explanation buried in a 45-minute recording usually means scrubbing through the whole thing.

ClipMind AI is my attempt at fixing that: an end-to-end platform that takes a video, transcribes it, summarizes it at two levels of depth, detects the moments that actually matter, and surfaces all of it through a role-aware dashboard. The backend is built on FastAPI with a dual-database design (PostgreSQL for structured, relational data, MongoDB for transcripts and AI output), the frontend on Next.js 16 with React 19, and the AI pipeline combines Groq-hosted Whisper Large v3 for speech-to-text with an open-weight 20B-parameter model (openai/gpt-oss-20b) served through the same Groq API for summarization, key-moment extraction, and quiz/study-guide generation. The whole stack is containerized with Docker and deployed to Render, with PostgreSQL on Render and MongoDB on Atlas.

To ensure maximum data durability and resilience on the cloud, the architecture was intentionally designed with a decoupled storage system. By separating the local compute nodes from the persistent PostgreSQL and MongoDB layers, the platform guarantees that all AI insights remain perfectly intact independent of the container lifecycle. This advanced decoupling of storage and compute ensures a highly scalable and fault-tolerant production environment.

1. Introduction

1.1 Problem Statement

Educators post lecture recordings, teams record standups and reviews, researchers sit through hour-long talks. In almost every case, the useful information is compressed into a few minutes of the recording, and there's no easy way to know which few minutes without watching the whole thing. Existing video platforms handle playback well, but they don't help a viewer understand what a video contains before committing time to it, and they give educators no automated way to turn a lecture into a study guide.

1.2 Proposed Solution

I designed ClipMind AI as a pipeline that ingests a video the moment it's uploaded (or linked from YouTube), strips and processes its audio, transcribes it, and runs the transcript through a summarization model to produce both a short and a detailed summary. In parallel, the pipeline detects key moments and extracts keywords, so a viewer gets a searchable, timestamped, skimmable version of the video instead of a static file. Everything is exposed through role-specific dashboards so a Content Creator, Educator, Learner, and Administrator each see the tools relevant to them.

1.3 Target Users

The platform is aimed at content creators, students, educators, media organizations, businesses, researchers, and online learning platforms, essentially anyone who produces or consumes long-form video and needs to extract information from it faster than real time.

2. Objectives and Scope

I scoped the project around four concrete objectives:

- Intelligent video processing, accept both local .mp4 uploads and YouTube URLs, and extract audio natively using FFmpeg and yt-dlp.
- AI integration, use Groq's LPU inference for both Whisper (speech-to-text) and LLM-based summarization, chosen specifically for its low latency.
- Role-based access control, give each of the four user types (Content Creator, Educator, Learner, Administrator) a distinct, purpose-built set of capabilities.
- Cloud scalability, containerize every service with Docker so the same stack runs identically in local development and in the cloud.

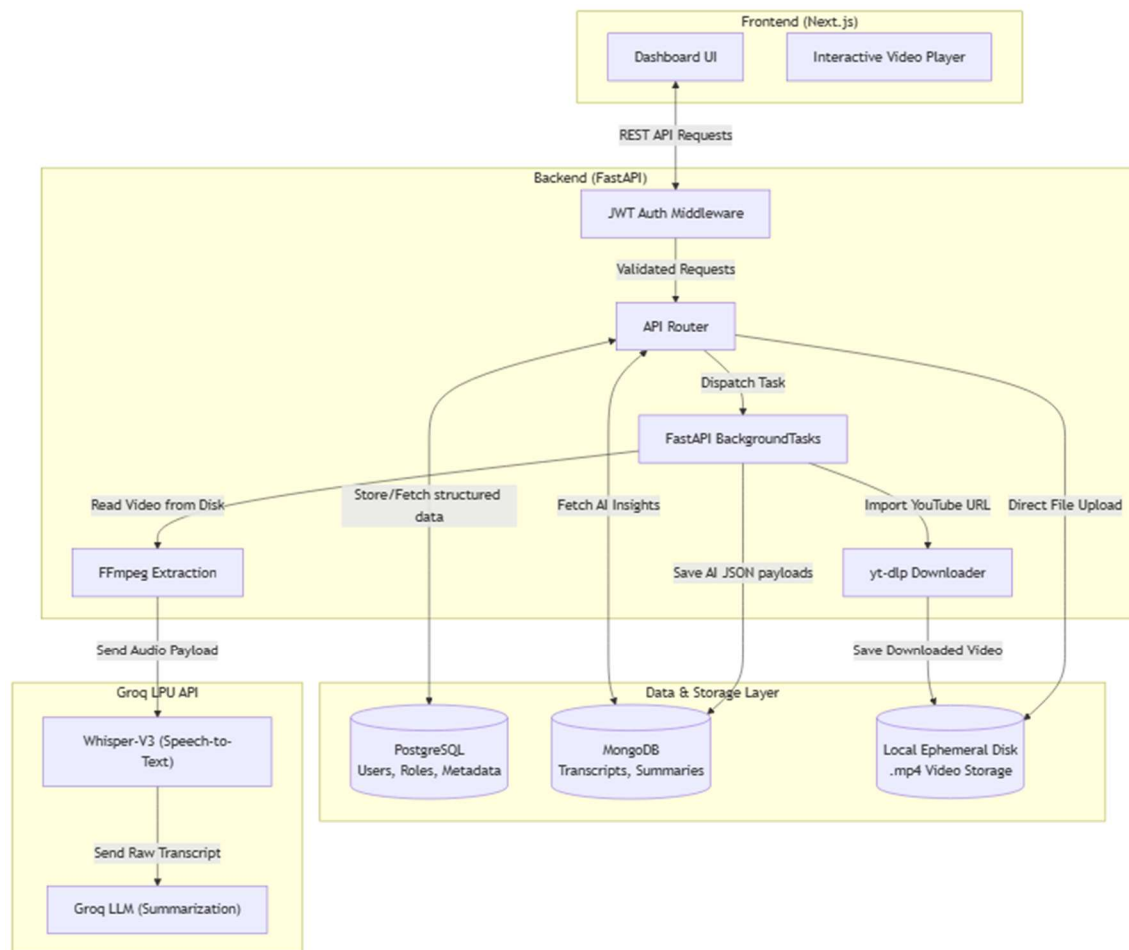


Figure 1. Overview of the entire system architecture for the ClipMind AI platform

3. System Architecture

ClipMind AI is organized into six layers so that each concern, client, gateway, AI processing, analytics, storage, and infrastructure, can be reasoned about and scaled independently. The client layer is the Next.js web app, every request from it passes through an API gateway layer inside FastAPI that handles JWT authentication, routing, role-based authorization, and rate limiting before it ever reaches business logic.

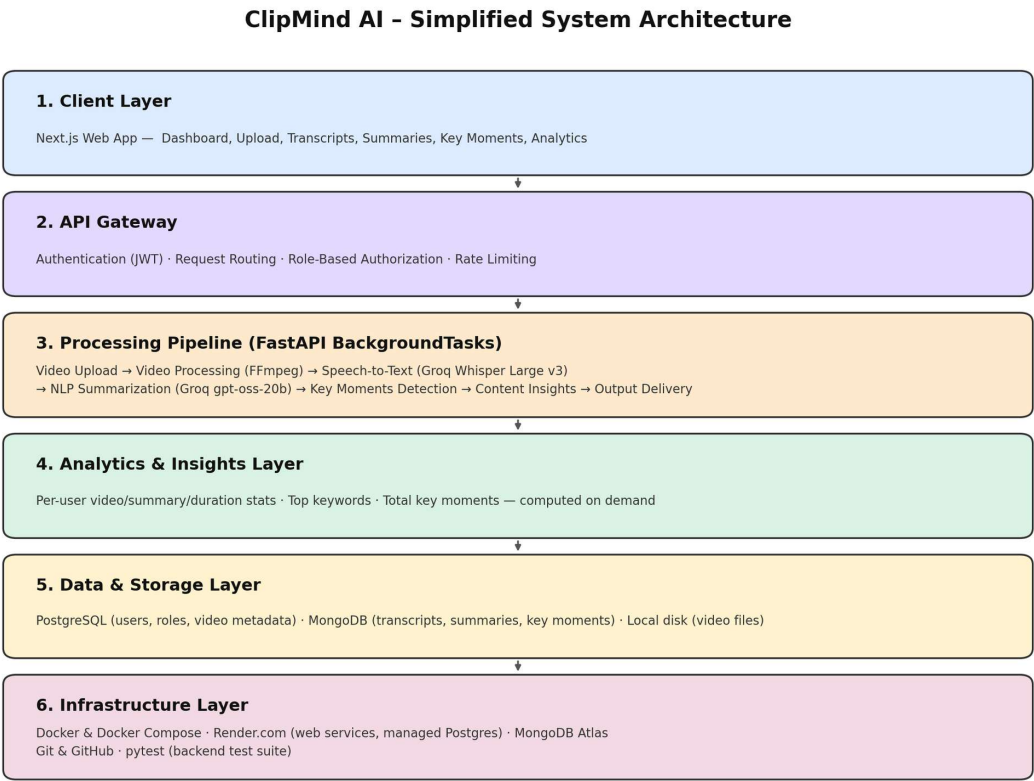


Figure 2. Simplified, layered system architecture, depicting client, API gateway, processing pipeline, analytics, storage, and infrastructure.

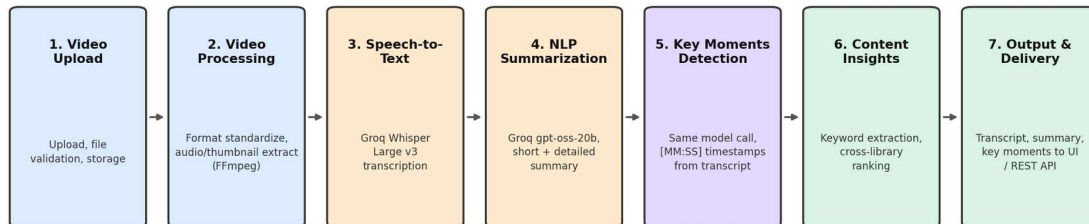
Underneath the gateway sits the processing pipeline: a set of asynchronous job workers, implemented with FastAPI BackgroundTasks, that run each video through the seven-stage media processing workflow described in Section 4. Results roll up into an analytics and insights layer that aggregates watch time, summary quality, and topic trends for the dashboards, while the data and storage layer splits responsibilities between PostgreSQL (users, roles, video metadata) and MongoDB (transcripts, summaries, key moments, the large, semi-structured AI output). The infrastructure layer is Docker-based, with the containers currently deployed on Render.

4. Media Processing Workflow

Once a video is uploaded, it moves through a sequential, asynchronously queued pipeline. Each stage writes its output to the database or object storage and updates the video's processing status, so the frontend can poll

for progress and show the user exactly where their video is, queued, processing, transcribed, summarized, or completed.

ClipMind AI - Media Processing Workflow



Each stage runs within a single asynchronous FastAPI BackgroundTask; status is tracked per video ID and polled by the frontend.

Figure 3. The seven-stage media processing pipeline, from upload to delivery.

4.1 Stage Breakdown

- **Video Upload:** The client uploads the file, the backend validates format and size and stores the raw video, capturing metadata such as title, description, and tags.
- **Video Processing:** FFmpeg standardizes the format and resolution, extracts the audio track, and generates a keyframe thumbnail.
- **Speech-to-Text:** The extracted audio is sent to Groq's whisper-large-v3 endpoint to produce a time-aligned, verbose-JSON transcript with per-segment start/end timestamps.
- **NLP Summarization:** The timestamped transcript is passed to openai/gpt-oss-20b (an open-weight 20B model served on Groq's LPU hardware) in a single structured-JSON call that returns a short summary, a detailed summary, key moments, and keywords together.
- **Key Moments Detection:** The same summarization call asks the model to extract 3–7 key moments, each anchored to an exact [MM:SS] timestamp pulled from the transcript rather than invented.
- **Content Insights:** The model returns 5–10 keywords per video in the same pass, which the analytics endpoint aggregates and ranks across a user's whole library.
- **Output & Delivery:** The transcript, summaries, key moments, and insights are made available through the dashboard and the REST API.

An early design decision worth noting during Milestone 1 planning was that I scoped the summarization stage around open-source BART/T5 models. Once I started benchmarking latency in Milestone 2, I moved summarization onto Groq's cloud inference instead, specifically an open-weight 20B model (openai/gpt-oss-20b), not a hosted LLaMA checkpoint. The evaluation notebook times this call directly against the live API and consistently comes back around 1–2 seconds for a multi-paragraph summary plus key moments, which was the difference between a pipeline that feels instant and one that feels like a batch job.

Two implementation details worth calling out because they aren't in the original design docs: the Groq client supports multiple API keys and automatically rotates to the next key on a rate-limit or authentication error

(api_client.py), so a single exhausted key doesn't take the pipeline down, and there's a standalone clip-extraction endpoint (POST /api/video/{video_id}/extract-clip) that uses FFmpeg to cut a start–end range out of an existing video and register it as a new video record, effectively turning a detected key moment into its own shareable clip.

5. Database Design

5.1 users (PostgreSQL)

Field	Type	Description
id	Integer, PK	Auto-increment primary key.
name	String	Full display name.
email	String, unique	Login identifier.
hashed_password	String	bcrypt hash via passlib, verified against JWTs issued by python-jose.
role	String	content_creator learner educator administrator.
is_active	Boolean	Defaults to true, reserved for account suspension.
created_at	DateTime	Account creation time (UTC).

5.2 videos (PostgreSQL)

Field	Type	Description
id	Integer, PK	Auto-increment primary key.
owner_id	Integer	Uploading user's id.
filename	String	Original filename, combined with the id to form the on-disk path.
title / description / tags	String	User-supplied metadata.
status	String	uploaded processing completed failed.
visibility	String	private public, controls whether learners can browse it.
classroom_id	Integer, nullable	Set when a video is attached to a classroom.
created_at	DateTime	Upload timestamp (UTC).

Notably, there's no duration, file size, or format column, those were part of the original design but weren't needed once the frontend started reading video metadata directly from the streamed file. The physical file lives on disk at uploads/{id}_{filename}, served back to the frontend through a dedicated GET /api/video/stream/{video_id} endpoint rather than a stored URL.

5.3 classrooms & classroom_students (PostgreSQL)

A feature that isn't in the original planning documents at all: educators can create a classroom, which generates a unique 6-character join code (classrooms.code), learners join with that code, creating a row in classroom_students that links a user to a classroom. Educators can toggle is_accepting_students to close enrollment, and videos can be scoped to a classroom via videos.classroom_id.

5.4 Supporting tables (PostgreSQL)

Table	Fields	Purpose
bookmarks	id, user_id, video_id, created_at	A learner toggles a bookmark on/off per video.
learning_history	id, user_id, video_id, watched_at	One row per watch event, used to compute unique viewers and recent activity.
audit_logs	id, action, user_id, target_id, details, timestamp	Written on registration, role changes, classroom creation/joins, and admin actions, read back through GET /api/admin/audit-logs.

5.5 MongoDB collections

MongoDB holds three collections inside a clipmind_ai database, accessed asynchronously through Motor: transcripts (one document per video_id, holding segments, a list of {start, end, text}, and the concatenated full_text), summaries (one document per video_id, holding summary, short_summary, key_moments, and keywords, all four are written together in a single AI call rather than being separate tables), and learning_materials (keyed by video_id and type, storing the generated quiz or study_guide JSON for educators). There's no separate keywords or analytics collection, the /api/analytics endpoint computes keyword frequency and total key-moment counts on the fly, by scanning a user's summaries documents at request time.

5.6 Relationships Summary

- users 1, N videos (owner_id), a user can upload many videos.
- videos 1, 1 transcripts; videos 1, 1 summaries; & in MongoDB, one document per video, by video_id.
- classrooms 1, N classroom_students, classrooms 1, N videos, an educator's classroom holds many enrolled learners and many videos.
- users 1, N bookmarks, users 1, N learning_history, many-to-many between users and videos.
- users 1, N audit_logs, every administrative or content action is logged against the acting user.

6. Roles & Core Modules

Role Based Access Control (RBAC) has been implemented end to end: the same login screen routes each of the four roles into a different dashboard shell, and the API gateway enforces the same boundaries server-side.

6.1 Content Creator

Uploads and manages videos, generates transcripts and AI summaries, reviews detected key moments and highlights, downloads transcripts and summaries, and views per-video content analytics.
Purpose: create and manage summarized content for an audience.

6.2 Learner

Views uploaded videos, reads generated summaries, browses timestamped key moments, searches within transcripts, and bookmarks summaries or highlights while the platform tracks learning history. Learner can also access the study guides, the quizzes or any other study materials that have been shared in their respective classrooms by their respective educators. They can even download notes or play the quizzes to test themselves.

Purpose: consume educational content efficiently.

6.3 Educator

Creates classrooms with a shareable join code, uploads lecture videos into a classroom, edits AI-generated transcripts (PUT /api/educator/video/{id}/transcript), generates quizzes and study guides on demand from the transcript, and pulls per-classroom analytics, total student views, bookmarks, and unique students. Educators can also generate study guides or notes, as well as quizzes for the learners for any particular video.
Purpose: turn long lecture content into concise, reusable learning material and track whether students are actually engaging with it.

6.4 Administrator

Manages users and roles, deletes users or videos platform-wide, views system-wide stats and every video regardless of owner, and reads the full audit log of registrations, role changes, and classroom activity.
Purpose: keep the platform operating securely and reliably.

6.5 Classroom System

This module isn't in the original project plan but is fully implemented, as an additional feature or USP (unique selling point) of the project: an educator creates a classroom and gets a random 6-character code, a learner submits that code to join, which is recorded in classroom_students. The educator can close enrolment at any point, list enrolled students, remove a student, and see the videos and per-classroom analytics scoped to that classroom, versus a learner, who only sees the classroom's videos and can leave.

6.6 Module Summary

- User Management: Registration, login, profile management (PUT /api/auth/me), RBAC, activity history.
- Video Upload: Local file upload and YouTube link ingestion (yt-dlp), file validation, streaming playback, visibility toggling, and deletion.
- Transcript Generation: Groq Whisper speech-to-text conversion, transcript storage, and educator-side editing.
- Video Summarization: Short and detailed AI summaries, content abstraction, generated in the same model call as key moments and keywords.
- Key Moments Detection: Timestamp generation anchored to real transcript timestamps, plus a dedicated clip-extraction endpoint to turn a highlight into its own video.

- **Analytics Dashboard:** This has per-user video/summary/duration stats and top keywords, that have been computed on demand, from MongoDB.
- **Classroom Management:** Join-code generation, enrolment, per-classroom video scoping and analytics.
- **AI Processing:** Groq-hosted speech recognition and LLM summarization with automatic API-key rotation on rate limits.

7. UI/UX Design

Before writing any frontend code, I sketched low-fidelity wireframes for the three core screens to lock down layout and navigation. All three are built mobile-responsive first, using React/Next.js with Tailwind CSS.

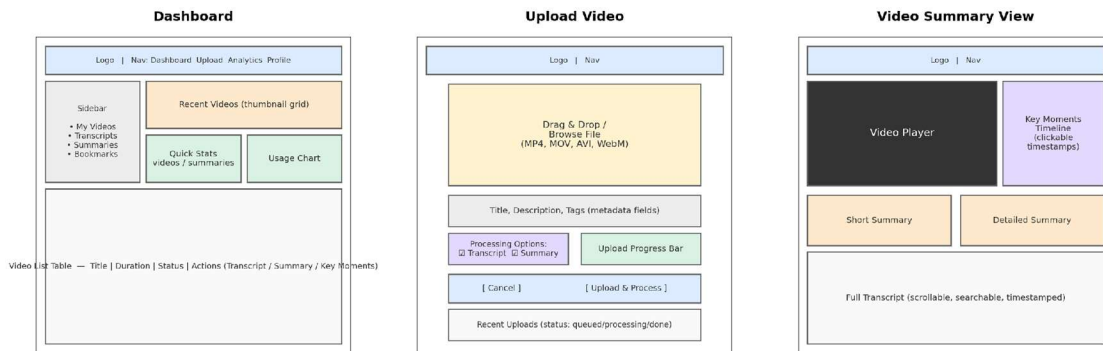


Figure 4. Low-fidelity wireframes, Dashboard, Upload, and Video Summary View.

- **Dashboard:** sidebar navigation, a recent-videos grid, quick stats with a usage chart, and a full video list table with per-row actions for transcript, summary, and key moments.
- **Upload:** drag-and-drop uploader with metadata fields, processing-option toggles (transcript/summary), a progress bar, and a recent-uploads status list.
- **Video Summary View:** a video player paired with a clickable key-moments timeline, short and detailed summary panels, and a scrollable, searchable, timestamped transcript below.

Navigation itself is role-aware: Content Creators land on upload and analytics, Learners get a read-only library with bookmarking, Educators get sharing and classroom analytics, and Administrators get a separate management console. Processing status is shown consistently as queued / processing / transcribed / summarized / completed / failed across both the Dashboard and Upload screens, so a user always knows where their video is in the pipeline.

8. API Reference

The backend exposes a REST API through FastAPI, which also generates interactive documentation at /docs (Swagger UI), available live at the deployed backend URL. Eight routers are mounted in main.py, each under its own prefix. The tables below reflect the endpoints as they exist in the current codebase.

8.1 Authentication (/api/auth)

Endpoint	Description
POST /register	Registers a new user (role defaults to content_creator) and returns a JWT access token.
POST /login	Authenticates via OAuth2 form data and returns a JWT access token.
GET /me	Returns the authenticated user's name, email, and role.
PUT /me	Updates display name and/or password after re-verifying the current password.

8.2 Video (/api/video)

Endpoint	Description
POST /upload	Uploads a local video file (multipart form) with title, description, tags, and optional classroom_id.
POST /youtube	Downloads a YouTube URL via yt-dlp and registers it as a video.
GET / and GET /{video_id}	Lists the authenticated user's videos or fetches one by id.
GET /stream/{video_id}	Streams the raw video file back to the player.
DELETE /{video_id}	Deletes a video and its file.
POST /{video_id}/process	Dispatches the AI pipeline as a FastAPI BackgroundTask.
PUT /{video_id}/visibility	Toggles a video between private and public.
POST /{video_id}/extract-clip	Cuts a start–end range out of the video with FFmpeg and saves it as a new video.

8.3 AI Results (/api/insights)

Endpoint	Description
GET /transcript/{video_id}	Returns the stored transcript segments and full text from MongoDB.
GET /summary/{video_id}	Returns the short/detailed summary, key moments, and keywords from MongoDB.
GET /materials/{video_id}	Returns generated quizzes and/or study guides for that video.

8.4 Educator, Learner, Classroom & Admin

Router	Representative endpoints
/api/educator	PUT video/{id}/transcript (edit), POST video/{id}/materials (generate quiz/study guide), GET analytics.
/api/learner	GET videos/public, POST/GET bookmark(s), POST/GET history.
/api/classroom	POST create, POST join, GET educator/learner, PUT {id}/toggle, GET {id}/students, GET {id}/analytics.
/api/admin	GET users, PUT users/{id}/role, DELETE users/{id}, GET system-stats, GET audit-logs, GET/DELETE videos.

8.5 Analytics (/api/analytics)

A single GET / endpoint returns the authenticated user's total videos, processed-video count, summaries generated, total key moments, top 5 keywords by frequency, and total processed duration in minutes, computed on the fly from PostgreSQL video counts and MongoDB summary/transcript documents rather than from a pre-aggregated table.

9. Tools & Tech Stack

9.1 Languages & Frameworks

Layer	Stack
Backend	Python 3.11, FastAPI, SQLAlchemy, Motor (async MongoDB driver)
Frontend	TypeScript, React 19, Next.js 16 (App Router, standalone build)
Styling	Tailwind CSS v4, Framer Motion, Recharts, Lucide React

9.2 Databases

Database	Role
PostgreSQL	Relational store for users, roles, classrooms, video metadata, hosted on Render.
MongoDB	Document store for transcripts, summaries, key moments, hosted on MongoDB Atlas (M0, Asia region).

9.3 AI & Video Processing

- Speech-to-Text: whisper-large-v3, served through the Groq LPU API for low-latency inference.
- Summarization & intelligence: openai/gpt-oss-20b via the Groq chat completions API, called once per video to produce the short summary, detailed summary, key moments, and keywords together as structured JSON, called again per request for educator quizzes and study guides.
- Resilience: the Groq client (ai/api_client.py) supports multiple API keys and automatically rotates on RateLimitError or AuthenticationError.
- Video/audio: ffmpeg-python for thumbnail extraction, audio extraction, and clip cutting, yt-dlp for YouTube ingestion.

9.4 DevOps & Tooling

- Containerization: Docker and Docker Compose; multi-stage builds for both frontend and backend.
- Hosting: Render.com (web services, managed Postgres) and MongoDB Atlas.
- Testing: pytest for the backend API and pipeline, Jupyter notebooks for AI benchmarking.
- Version control: Git and GitHub.

10. Milestone-Wise Execution

I structured the eight-week build around four two-week milestones, each closing with a working, demoable increment rather than a partial feature.

Milestone	Weeks	Focus	Status
1	1–2	Architecture design, DB schema, JWT auth + RBAC, video upload, FFmpeg pipeline	Completed
2	3–4	Whisper transcription via Groq, transcript storage, AI summarization (brief + detailed)	Completed
3	5–6	Key moment timestamp extraction, keyword extraction, analytics dashboard with Recharts	Completed
4	7–8	pytest suite, evaluation notebook, Dockerization, deployment to Render, documentation	Completed

Each milestone's evaluation criteria were tied to a working deliverable: Milestone 1 required a functional authentication and upload flow, Milestone 2 required working Whisper integration and AI summary generation, Milestone 3 required a functional key-moments and analytics dashboard, and Milestone 4 required a fully deployed frontend and backend with a completed test suite and documentation, all four were met.

11. Deployment

ClipMind AI is fully containerized and deployed to the cloud. The frontend is a multi-stage Dockerfile using Next.js standalone mode to keep the runtime image small on an Alpine base, listening on 0.0.0.0:3000 and pointed at the backend through NEXT_PUBLIC_API_URL. The backend Dockerfile is built on python:3.11-slim with ffmpeg installed at the system level, exposes port 8000, and runs AI summarization through FastAPI BackgroundTasks so long-running jobs don't block or time out HTTP requests.

Component	URL / Host
Frontend (Next.js)	https://clipmind-ai-frontend.onrender.com/
Backend (FastAPI)	https://clipmind-ai-8hcx.onrender.com/
API Docs (Swagger)	https://clipmind-ai-8hcx.onrender.com/docs
PostgreSQL	Render managed Postgres, internal networking to backend
MongoDB	MongoDB Atlas, M0 cluster, Asia region

To run the full stack locally: clone the repository, ensure Docker Desktop is running, provide the required .env variables in backend/, and run docker-compose up --build. The frontend then serves at http://localhost:3000 and the backend at http://localhost:8000.

12. Local Demo Rationale

The application is production-architected and fully cloud-capable, but I run live demonstrations locally rather than against the hosted Render instance, for three specific, well-understood reasons rather than any gap in the application logic itself.

12.1 Ephemeral Storage

Render's free-tier web services use an ephemeral filesystem: when a container spins down from inactivity or restarts during a deploy, anything written to /app/uploads is wiped, even though the database still holds the video's metadata. That leaves a video's .mp4 file missing while its record still exists. The production fix is an S3-compatible object store (AWS S3 or Cloudflare R2), for the scope of this project, running the backend locally keeps the filesystem persistent for the demo.

12.2 YouTube IP Blocking

The YouTube-link import feature relies on yt-dlp, and YouTube actively throttles and blocks IP ranges belonging to known cloud data centers (AWS, GCP, Render) to prevent scraping. Requests from the hosted backend intermittently return HTTP 403. This is a network-level restriction, not an application bug, running locally on a residential IP avoids it entirely.

12.3 Cold Starts

Render's free-tier instances spin down after 15 minutes of inactivity, waking one back up (a “cold start”) can take up to 60 seconds, during which the Whisper/LLM pipeline can time out mid-request. A local FastAPI server stays permanently warm, so processing starts immediately.

13. Testing, Evaluation & Performance

13.1 Testing

The testing/ directory holds a pytest suite of roughly 700 lines across three files, test_all.py (the bulk of the API and pipeline coverage), test_auth.py, and test_video_pipeline.py, run against an isolated SQLite engine (conftest.py) so tests don't touch the production Postgres/MongoDB instances. Separately, ai/evaluation.ipynb calls the summarization function (_groq_summarize) directly against a sample transcript and times the round trip, specifically to benchmark AI latency outside the request/response cycle.

13.2 AI Performance

Switching summarization to the Groq LPU API was the single biggest performance win in the project: the evaluation notebook times a full summary-plus-key-moments generation at roughly 1–2 seconds per request, versus the multi-second-to-tens-of-seconds latency typical of self-hosted BART/T5 inference on CPU, which is what I'd originally scoped in the Milestone 1 design. Whisper transcription over Groq showed similarly favourable latency for typical lecture-length audio.

14. Conclusion & Future Scope

ClipMind AI set out to solve a problem I'd run into myself more times than I can count: sitting through a 45-minute lecture or standup recording just to find the two minutes that actually mattered. Over these eight weeks, that idea became a working, role-aware platform that takes a raw video and turns it into a transcript, a short and detailed summary, timestamped key moments, and a searchable analytics layer, all built on a stack (FastAPI, Next.js, PostgreSQL, MongoDB, Docker) that runs identically whether it's on my laptop or deployed to the cloud. Every objective I scoped at the start, intelligent video processing, low-latency AI

integration, role-based access control, and cloud scalability, was met, and two features that were never in the original plan at all, the classroom system and the clip-extraction endpoint, ended up being some of the most useful parts of the product.

Architecturally, the biggest lesson was about separation of concerns. Splitting storage by access pattern, relational data in PostgreSQL, large semi-structured AI output in MongoDB, made the rest of the system easier to reason about instead of harder, which wasn't obvious to me going in. Moving heavy AI calls off the request path with FastAPI BackgroundTasks was the other decision that paid for itself immediately: the difference between a pipeline that feels instant and one that feels like a batch job came down almost entirely to that one architectural choice, not to the AI models themselves.

Beyond the architecture, this project was the first time I had to own a system end to end, from database schema to deployment, rather than just a slice of one. That meant making calls I couldn't defer to someone else: benchmarking BART/T5 against Groq's hosted inference myself in Milestone 2 and switching when the latency numbers didn't lie, figuring out why the hosted backend was silently failing on YouTube imports and tracing it back to IP-level blocking rather than a bug in my own code, and deciding which "nice to have" features (classrooms, clip extraction) were worth the extra time and which weren't. I came out of it more comfortable reading a stack trace across three different services and more confident trusting a benchmark over an assumption.

The three production constraints I hit late, ephemeral storage on Render's free tier, YouTube's blocking of data-center IPs, and cold starts, aren't flaws in the application logic, but they're honest reminders that a system's design and its hosting environment are two different problems, and free-tier infrastructure will always be the harder of the two to reason around. That's exactly where the future work below picks up: it isn't a wishlist so much as a prioritized list of the constraints I understand well enough now to actually solve.

Future Enhancements

- Move video storage to AWS S3 or Cloudflare R2 to eliminate the ephemeral-storage constraint and enable true horizontal scaling on Render or any container host.
- Add multi-language transcript translation.
- Implement RAG so users can chat directly with a video's content instead of only reading a static summary.
- Add a residential-proxy or official YouTube Data API fallback for the import pipeline to remove the IP-blocking constraint in production.